

Vincenzo Lombardi

Matricola: 0001235528

Multimedia Data Management

SmartImageSearch: Content-Based Image Retrieval System

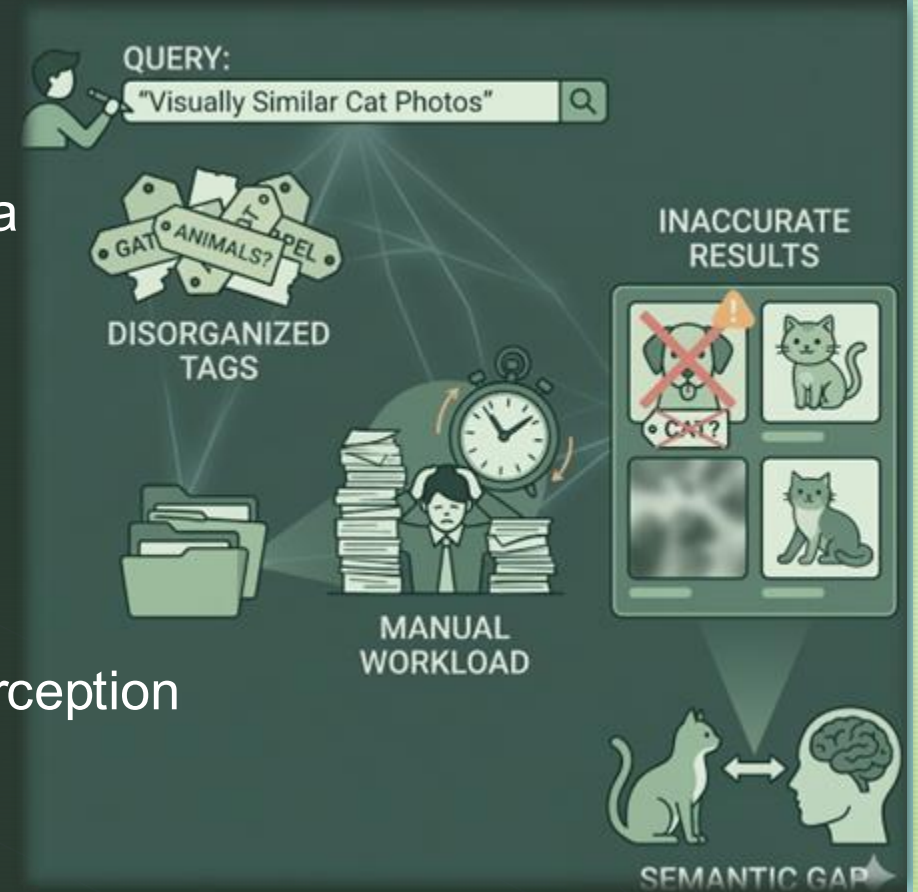
CONTENT:

- Problem
- Proposed Solution
- Multimedia Data Types
- MetadataSimilarity
- MeasuresUse
- Cases
- Benefits
- Conclusion



PROBLEM:

- Traditional image search relies on textual metadata
- Metadata can be incomplete or inaccurate
- Difficult to retrieve visually similar images
- Manual annotation is time-consuming
- Semantic gap between visual content and user perception



Current systems do not support effective visual search based on image content

- No similarity-based retrieval
- No content-based exploration
- Limited support for large image collections



LIMITATIONS OF CURRENT IMAGE RETRIEVAL SYSTEMS

TEXT-BASED QUERY (PRESENT)



Dependent on imprecise manual tagging and keyword mismatch.

MISSING CONTENT ANALYSIS (VISUAL FEATURES)

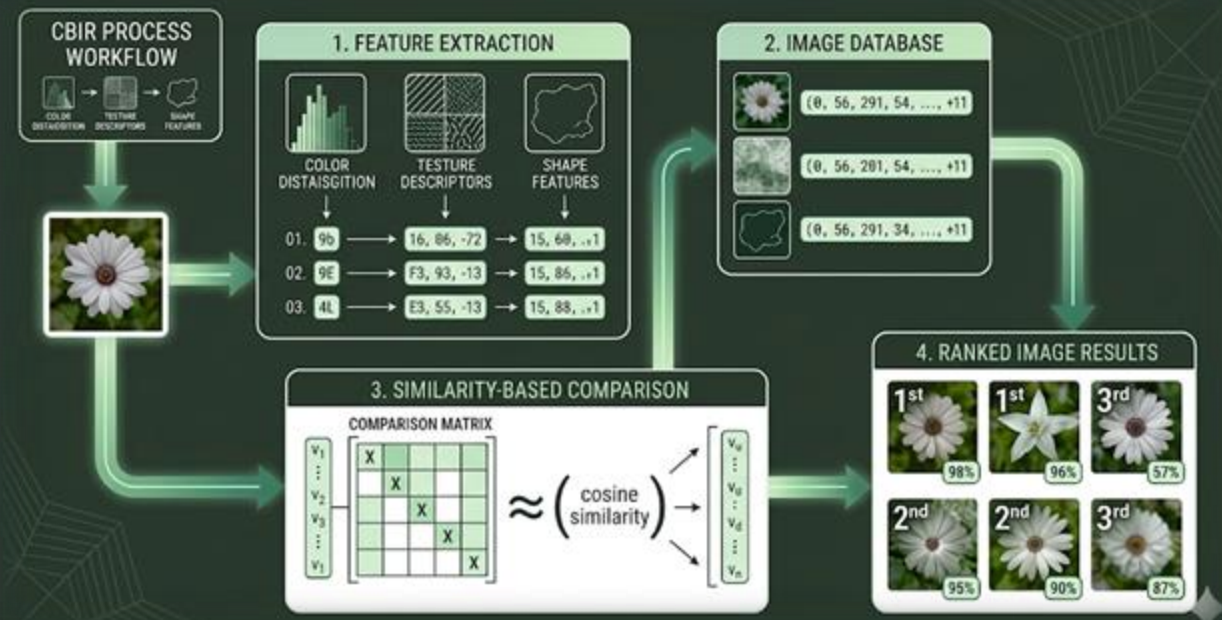


Visual details (color, texture, form) are ignored during retrieval.

PROPOSED SOLUTION: SmartImageSearch

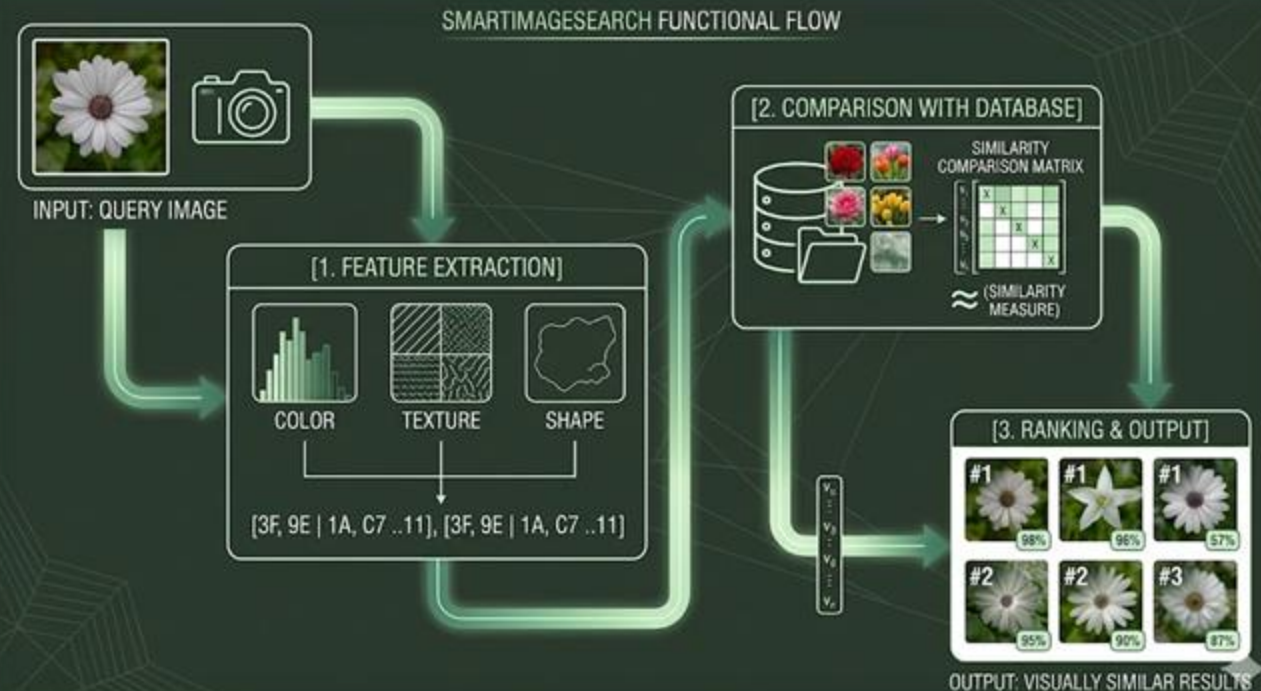
SmartImageSearch is a multimedia system designed to retrieve images based on visual similarity.

- Content-Based Image Retrieval (CBIR)
- Feature extraction from images
- Similarity-based comparison
- Ranked image results



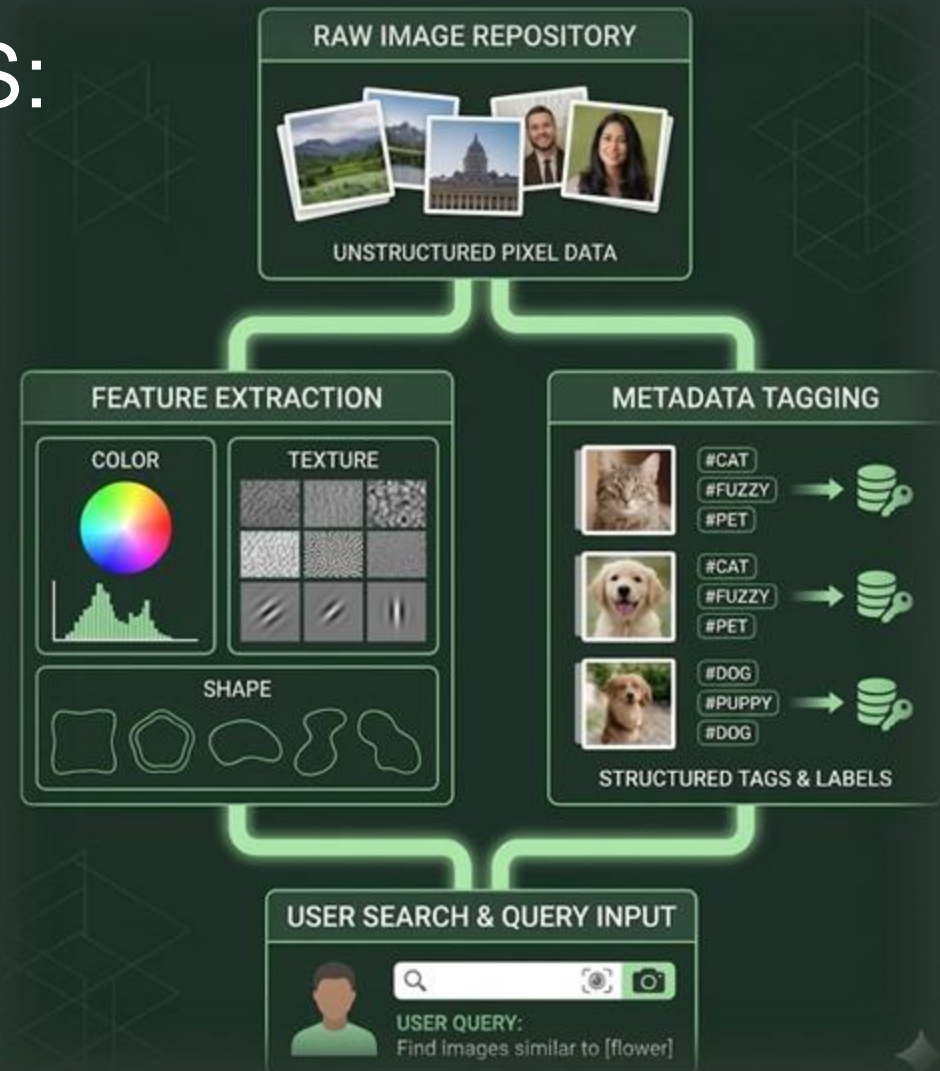
SYSTEM FUNCTIONALITY:

- Input: query image
- Feature extraction
- Comparison with database
- Ranking of similar images
- Output: visually similar results



MULTIMEDIA DATA TYPES:

- Image data (raw images)
- Visual features (color, texture, shape)
- Metadata (tags, labels)
- User queries



DATA REPRESENTATION:



UNSTRUCTURED DATA



- Raw images
- Visual content
- Pixel information



SEMI-STRUCTURED DATA



- Metadata (XML / JSON)
- Tags and labels
- Descriptive attributes



STRUCTURED DATA

Image ID	Category	File Size	Upload Date
NYC_001	Cityscape	5.2 MB	2023-10-27
NYC_002	Cityscape	5.2 MB	2023-10-28
NYC_003	Cityscape	5.2 MB	2023-10-28
NYC_004	Cityscape	5.2 MB	2023-10-29
...	...	...	...
...	...	...	...

- Structured Data
 - Image ID
 - Category
 - File size
 - Upload date

RELATIONAL DATA MODEL:

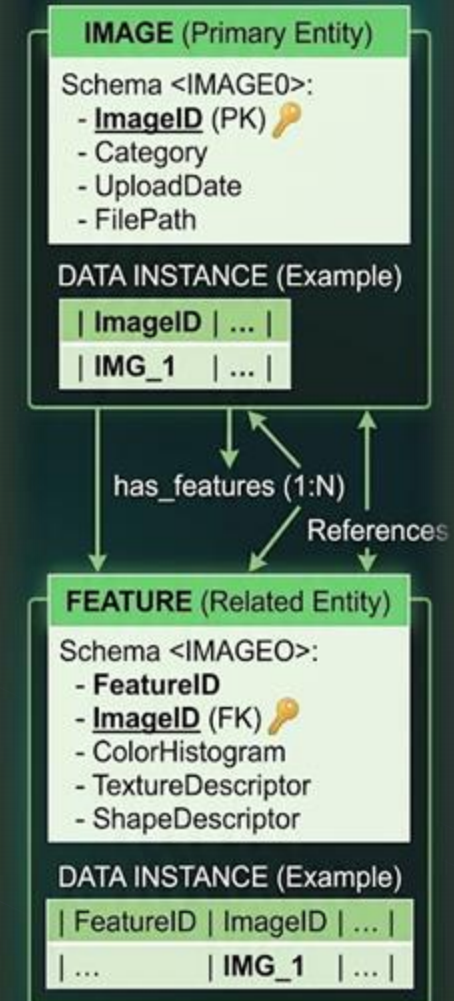


- Constraints:

ImageID → Primary Key

ImageID in Features → Foreign Key

DATABASE KEY CONSTRAINTS VISUALIZATION



ENTITY-RELATIONSHIP MODEL:



ENTITIES:



- Image
- Feature



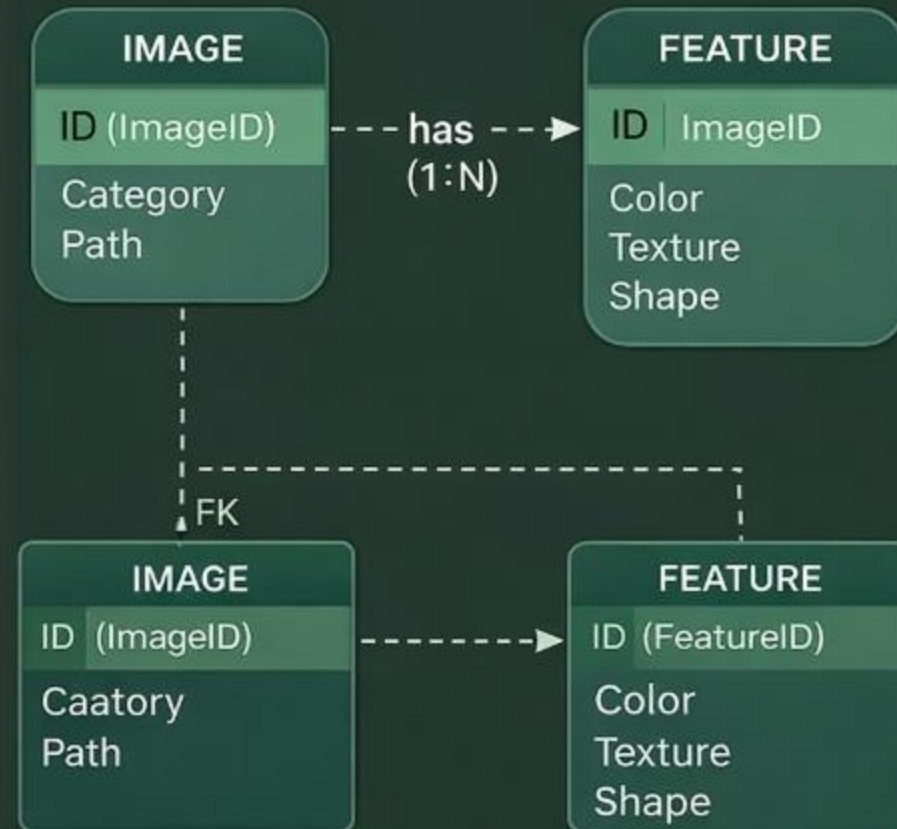
RELATIONSHIPS:

- Image $\rightarrow$ has $\rightarrow$ Feature (1:N)



ATTRIBUTES:

- Image: ID, Category, Path
- Feature: Color, Texture, Shape



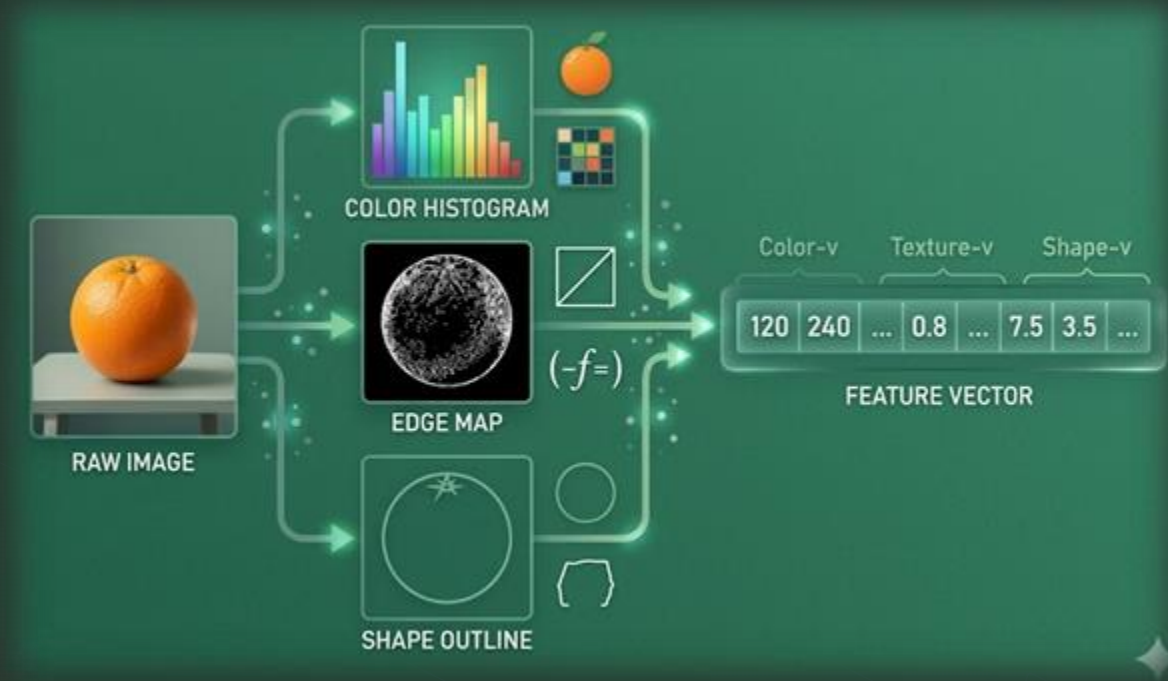
XML REPRESENTATION:

```
root@presentation:~#  
<Image id="001">  
  <Metadata>  
    <Category>animal</Category>  
    <Format>JPEG</Format>  
  </Metadata>  
  <Tags>  
    <Tag>dog</Tag>  
    <Tag>outdoor</Tag>  
  </Tags>  
</Image>
```

Example XML structure for a single image

FEATURE EXTRACTION PROCESS:

- Input:
 - raw image
 - Color histogram extraction
 - Texture extraction
 - Shape detection
- Output:
 - Feature vector



FEATURE VECTOR REPRESENTATION:

► Mapping Process:

- Image → vector:
- $f(I) = [c_1, c_2, \dots, t_1, t_2, \dots, s_1, s_2]$

► Feature Breakdown:

- Where:
 - c = color features
 - t = texture features
 - s = shape features

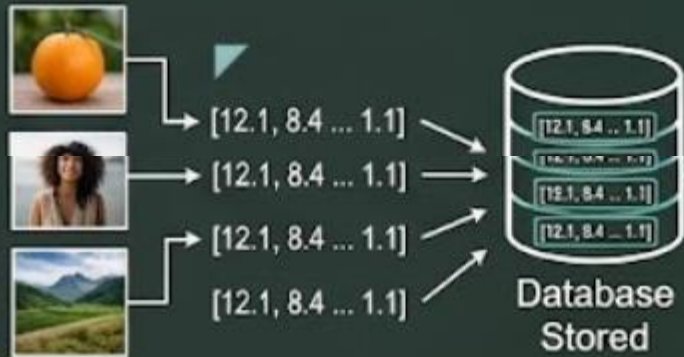
► Key Applications:

- Used for:
 - similarity computation
 - indexing



FEATURE INDEXING:

INPUT: Feature Vectors

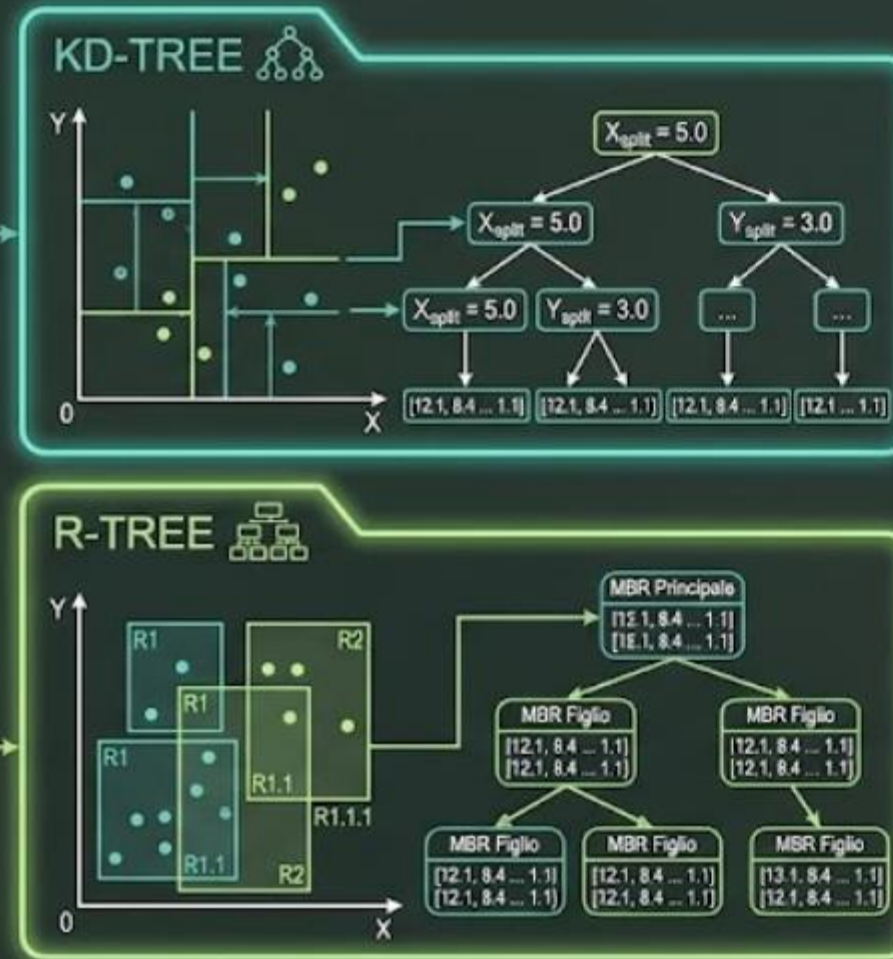


Mapping Process:

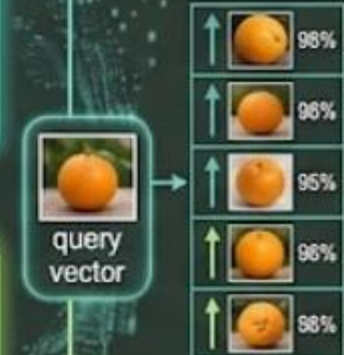
- Feature vectors stored in database
- Index structures:
 - KD-Tree
 - R-Tree

Key Applications:

- Supports fast similarity search
- similarity computation
- indexing



SIMILARITY SEARCH



(-f=)

TEXTUAL DOCUMENTS FOR RETRIEVAL:

In SmartImageSearch, textual retrieval is applied to image-associated text. Each image is represented by a short textual document.

Textual data includes:

- Tags
- Labels
- category
- Words
- short captions



DOCUMENT COLLECTION:

D1



dog · outdoor · park · grass · running

D2



cat · indoor · sofa · sleeping · home

D3



dog · beach · sunset · sea · sand

D4



car · street · city · traffic · road

D5



cat · garden · outdoor · tree · nature

D6



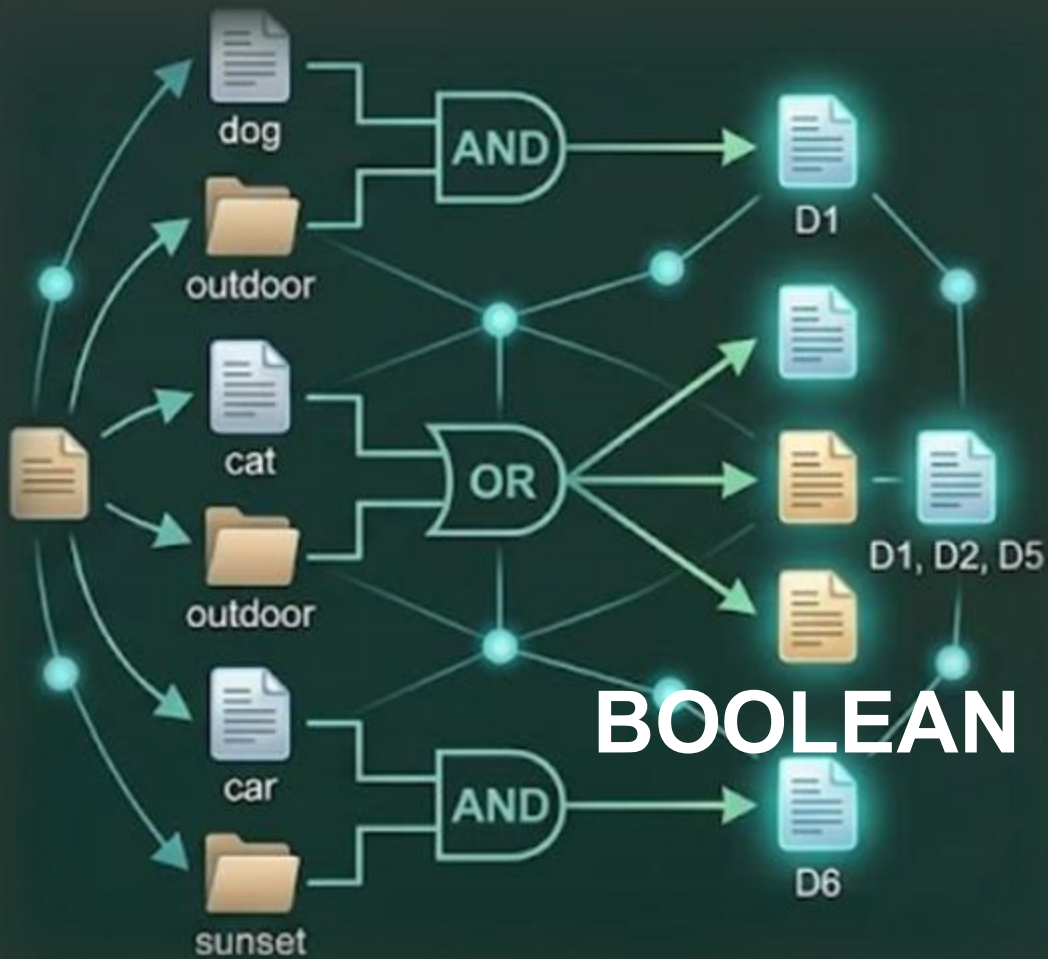
car · highway · sunset · road · travel

TERM-DOCUMENT MATRIX

Term	D1	D2	D3	D4	D5	D6
dog	1	0	1	0	0	0
cat	0	1	0	0	1	0
car	0	0	0	1	0	1
outdoor	1	0	0	0	1	0
sunset	0	0	1	0	0	1
road	0	0	0	1	0	1
beach	0	0	1	0	0	0
sofa	0	1	0	0	0	0

INVERTED INDEX:

Term	Documents
dog	D1, D3
cat	D2, D5
car	D4, D6
outdoor	D1, D5
sunset	D3, D6
road	D4, D6
beach	D3
sofa	D2



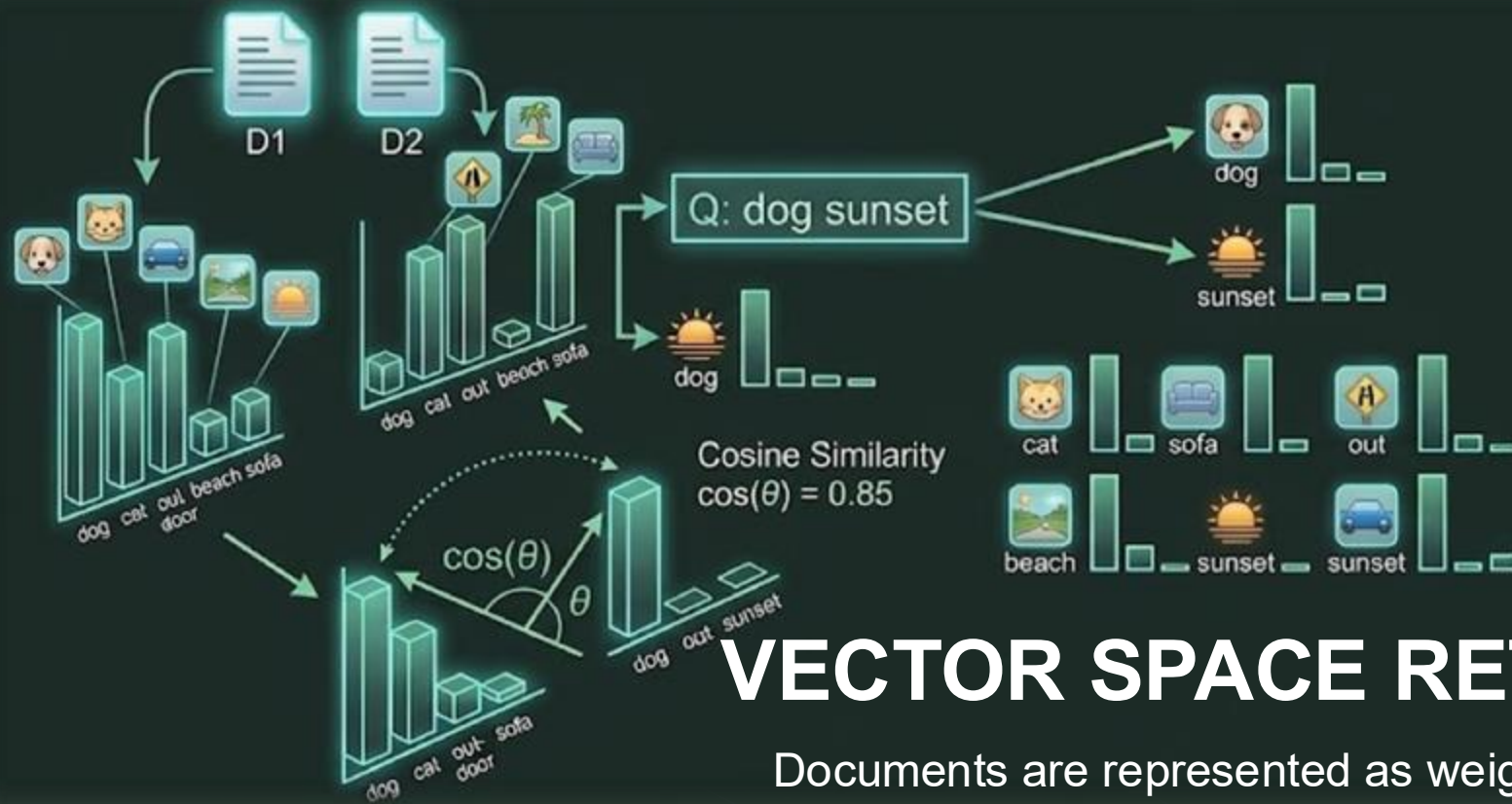
BOOLEAN RETRIEVAL QUERIES:

Q1: dog **AND** outdoor → D1

Q2: cat **OR** outdoor → D1, D2, D5

Q3: car **AND** sunset → D6

*Boolean model returns exact matches
Results are not ranked*



VECTOR SPACE RETRIEVAL:

Documents are represented as weighted term vectors

Query terms are also represented as vectors

Documents are ranked by similarity to the query

Similarity can be computed with cosine similarity

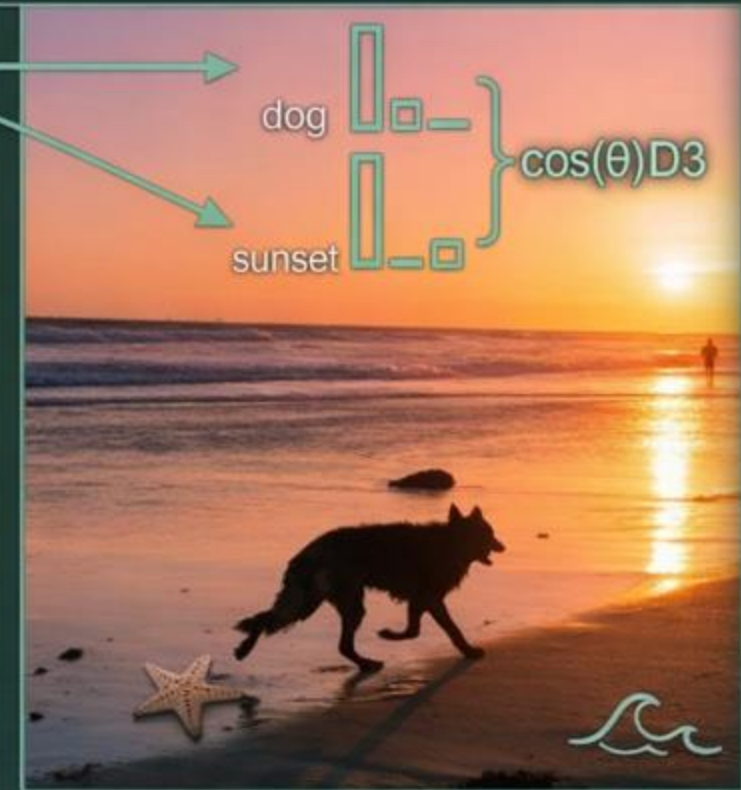
An example query: dog sunset

RANKED RESULTS EXAMPLE:

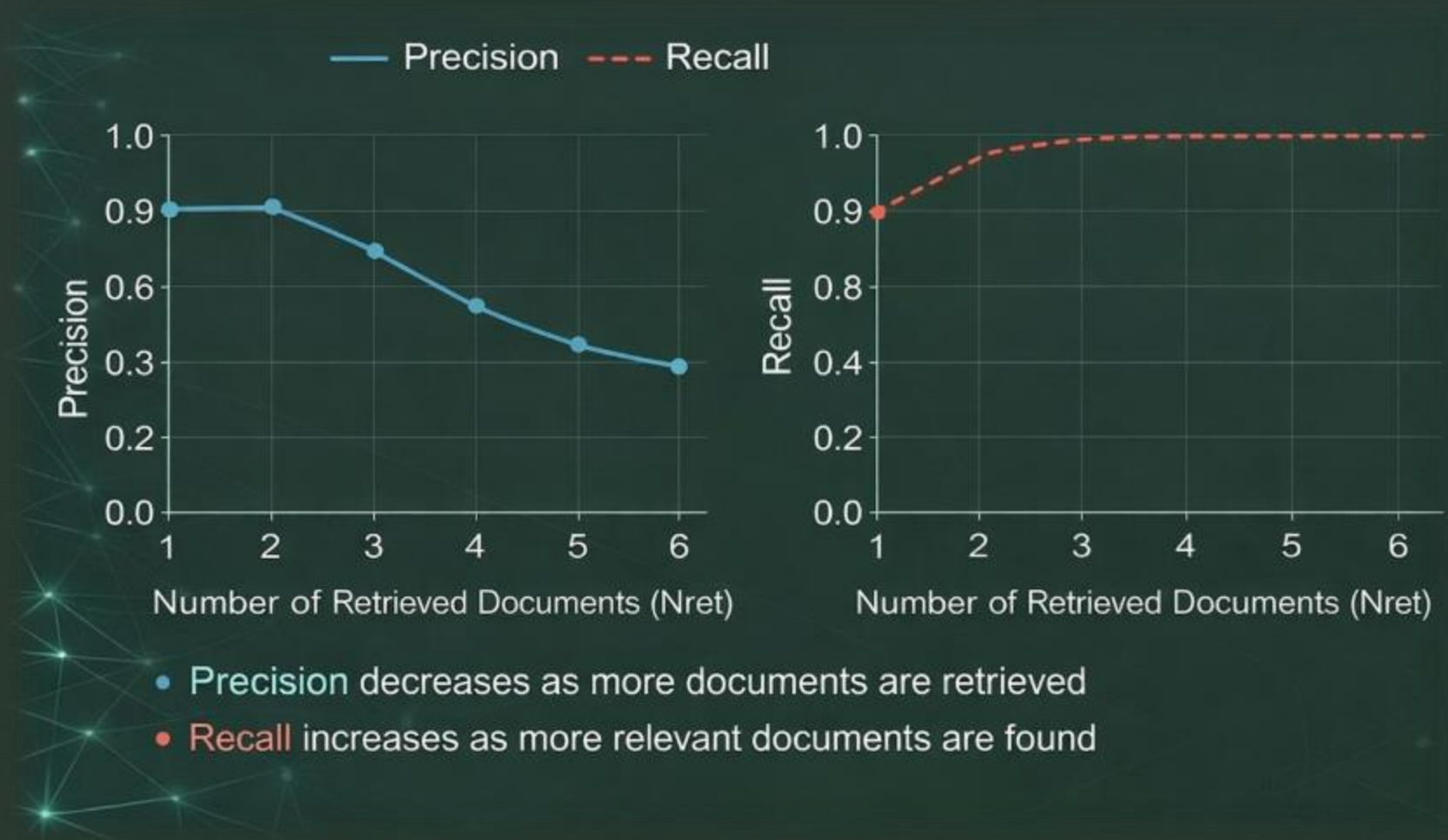
- Q = dog sunset
- D3 = dog beach sunset sea sand
- D1 = dog outdoor park grass running
- D6 = car highway sunset road travel
- D2 = cat indoor sofa sleeping home
- D5 = cat garden outdoor tree nature
- D4 = car street city traffic road

Relevant documents: D3, D1

D3 is the most relevant because it contains both query terms (dog, sunset)



▲ AVERAGE PRECISION AND RECALL VS Nret:



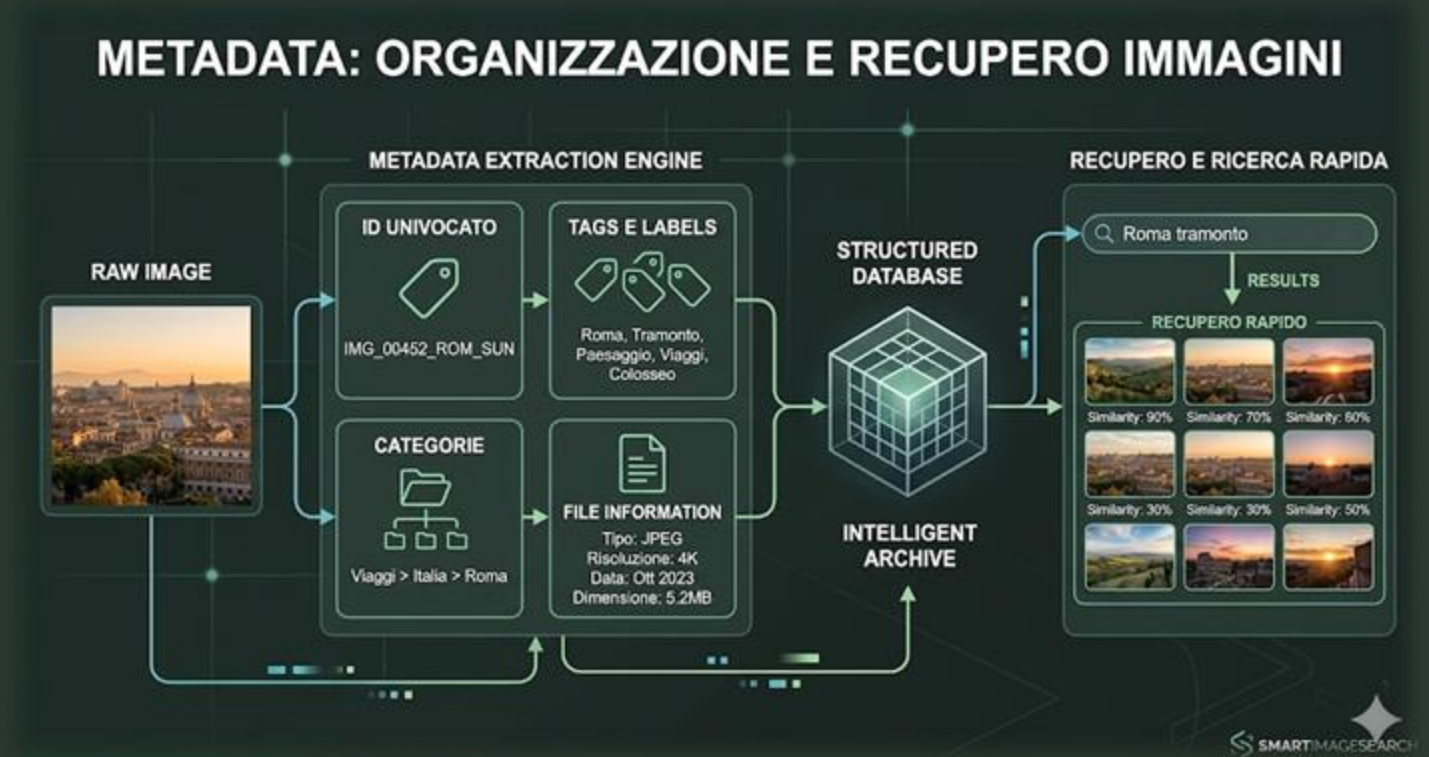
VISUAL EXAMPLES OF TEXTUAL INFORMATION:

▪		→	+ dog · outdoor · park · grass · running
→ ▪		→	+ cat · indoor · sofa · sleeping · home
→ ▪		→	+ car · street · city · traffic · road

METADATA

Metadata supports image organization and retrieval

- Image ID
- Tags / labels
- Categories
- File information



SIMILARITY MEASURES

- Distance between feature vectors
- Euclidean distance
- Cosine similarity
- Ranking based on similarity score

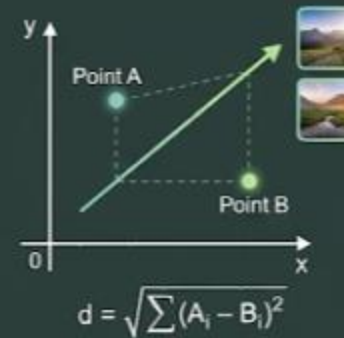
SIMILARITY MEASUREMENT PROCESS

Distance between Feature Vectors

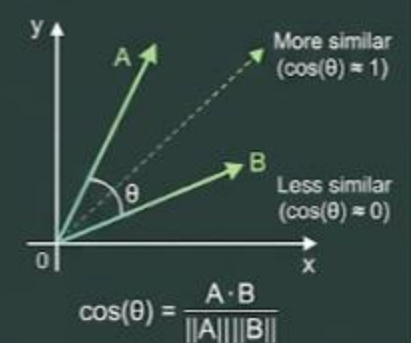


Euclidean vs. Cosine Distance

Euclidean distance (d)



Cosine similarity (cos(θ))



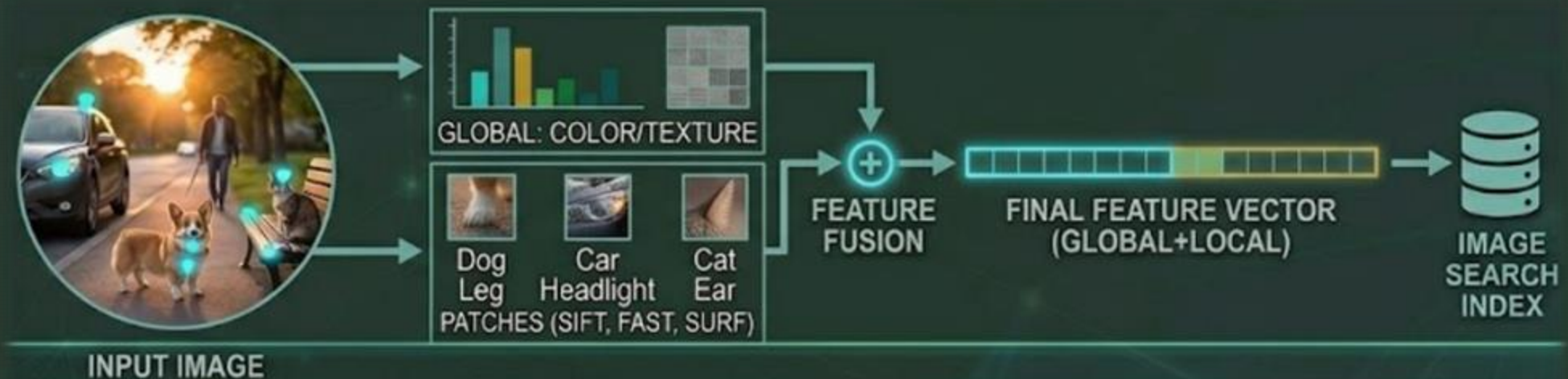
GLOBAL VS LOCAL FEATURES:

Global Features

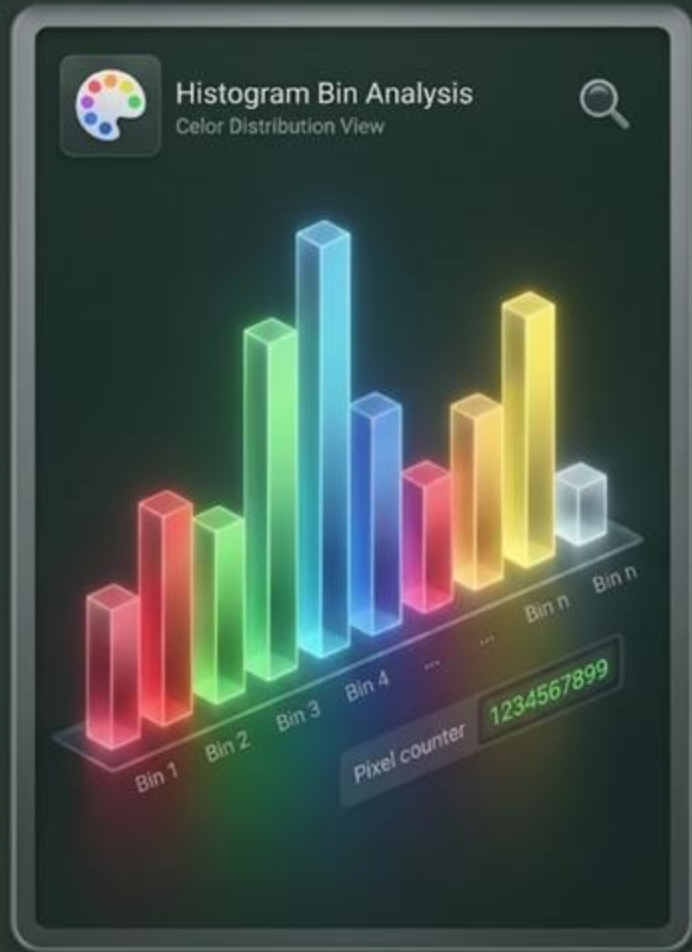
- Extracted from the entire image
- Capture overall visual properties
- Example: color distribution

Local Features

- Extracted from specific regions or keypoints
- Capture local patterns and structures
- Example: edges, corners



COLOR HISTOGRAM REPRESENTATION:



- Images are represented using color histograms
- A histogram describes the distribution of colors
- Each bin counts the number of pixels in a color range

$$H(I) = [h_1, h_2, \dots, h_n]$$

Where:

h_i = number of pixels in bin i

VISUAL EXAMPLE: COLOR HISTOGRAM



The histogram captures dominant colors (e.g., orange, blue)

SIMILARITY MEASURE

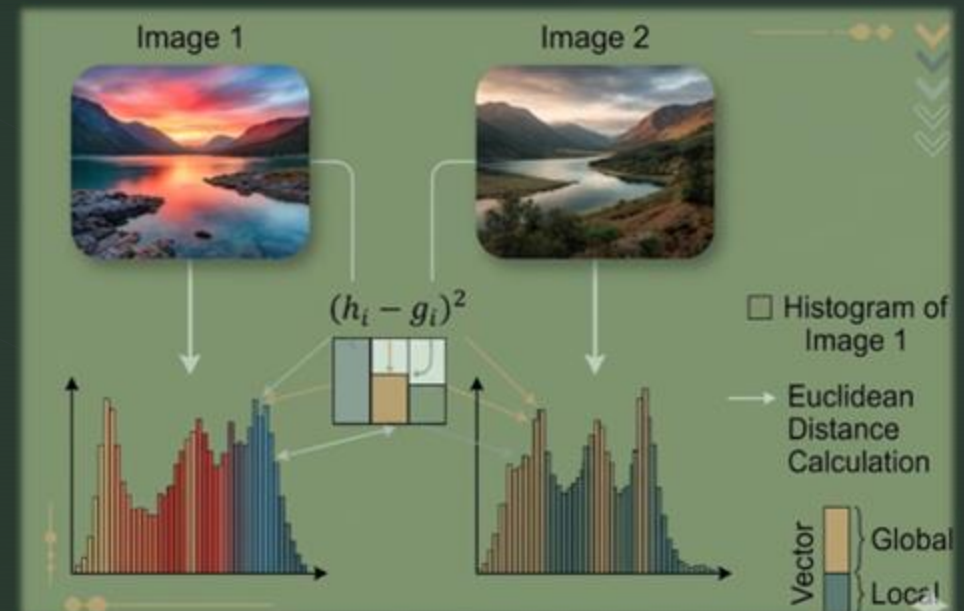


- Images are compared using histogram distances
- We use Euclidean distance:

$$d(I_1, I_2) = \sqrt{\sum_{i=1}^N (h_i - g_i)^2}$$

Where:

- h_i = histogram of image 1
- g_i = histogram of image 2



FEATURE COMPARISON EXAMPLE

- Image A → histogram HA
- Image B → histogram HB

Distance: $d(HA, HB)$

- Smaller distance → higher similarity
- Larger distance → lower similarity

FEATURE COMPARISON ILLUSTRATION

SCENARIO 1: HIGHER SIMILARITY (Smaller Distance)



$$d(HA, HB) = 0.15$$

Distance is small,
meaning histograms
are very similar.



✓ Similarity: 92%

SCENARIO 2: LOWER SIMILARITY (Larger Distance)



$$d(HA, HB') = 0.78$$

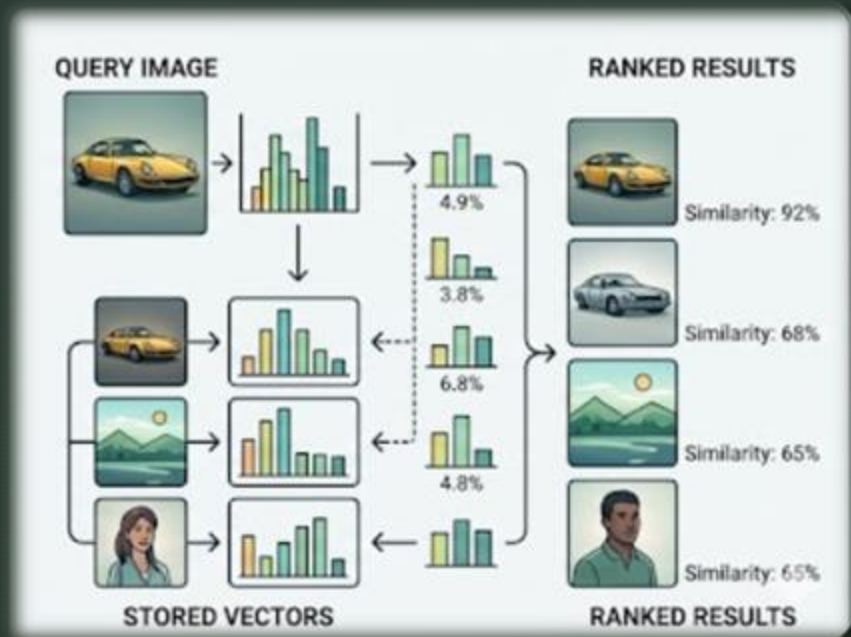
Distance is large,
meaning
histograms are
quite different.



✗ Similarity: 25%



FROM FEATURES TO RETRIEVAL



- Extract features from all images
- Store feature vectors in database
- Compare query image with stored features
- Rank images based on similarity

SYSTEM ARCHITECTURE OVERVIEW:

- Query image input
- Feature extraction module
- Feature database
- Similarity computation
- Ranking module
- Result visualization

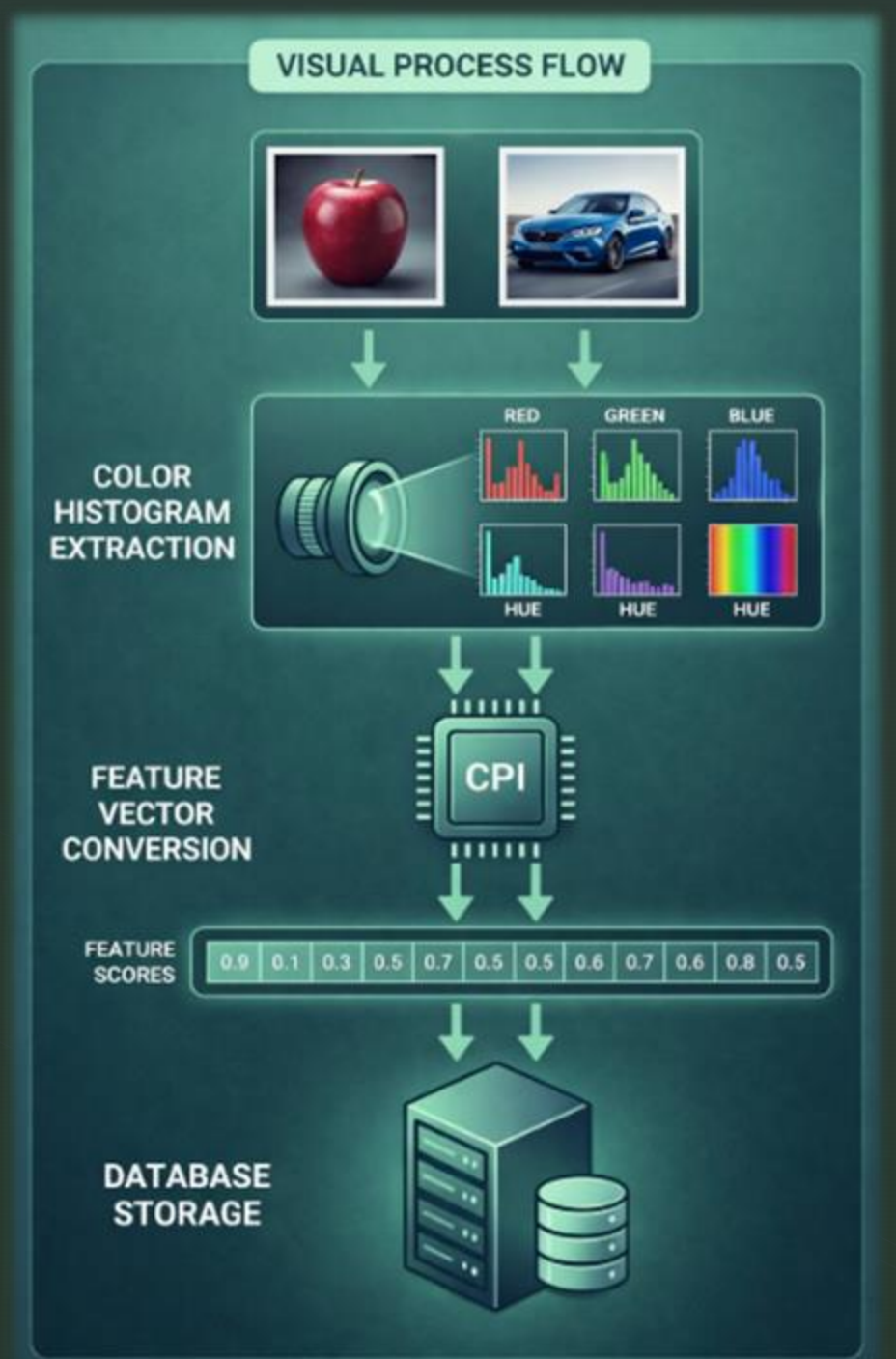


SYSTEM ARCHITECTURE DIAGRAM:



FEATURE EXTRACTION MODULE

- Extract color histograms
- Convert images into feature vectors
- Store features in database



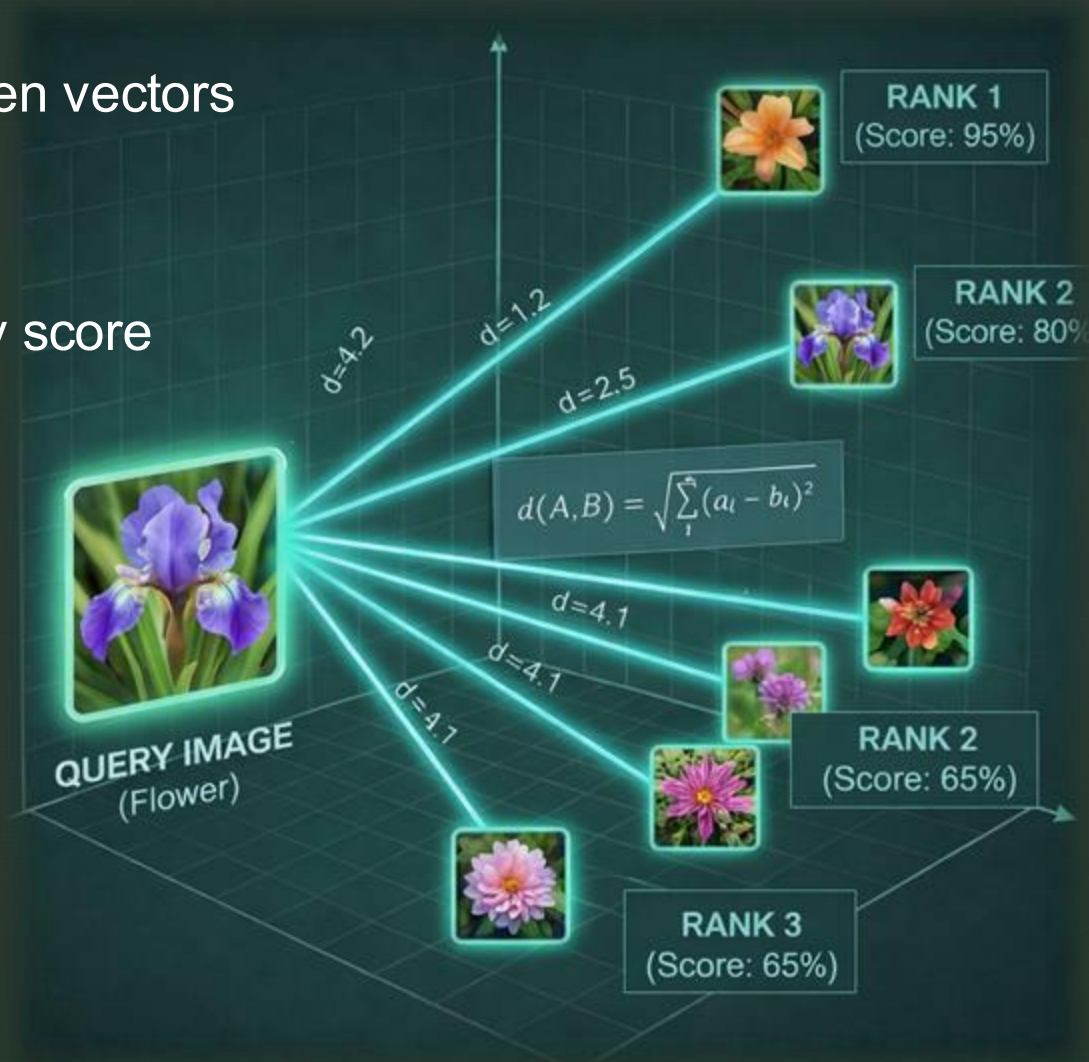
FEATURE DATABASE

- Stores feature vectors
- Linked to image IDs
- Supports fast retrieval



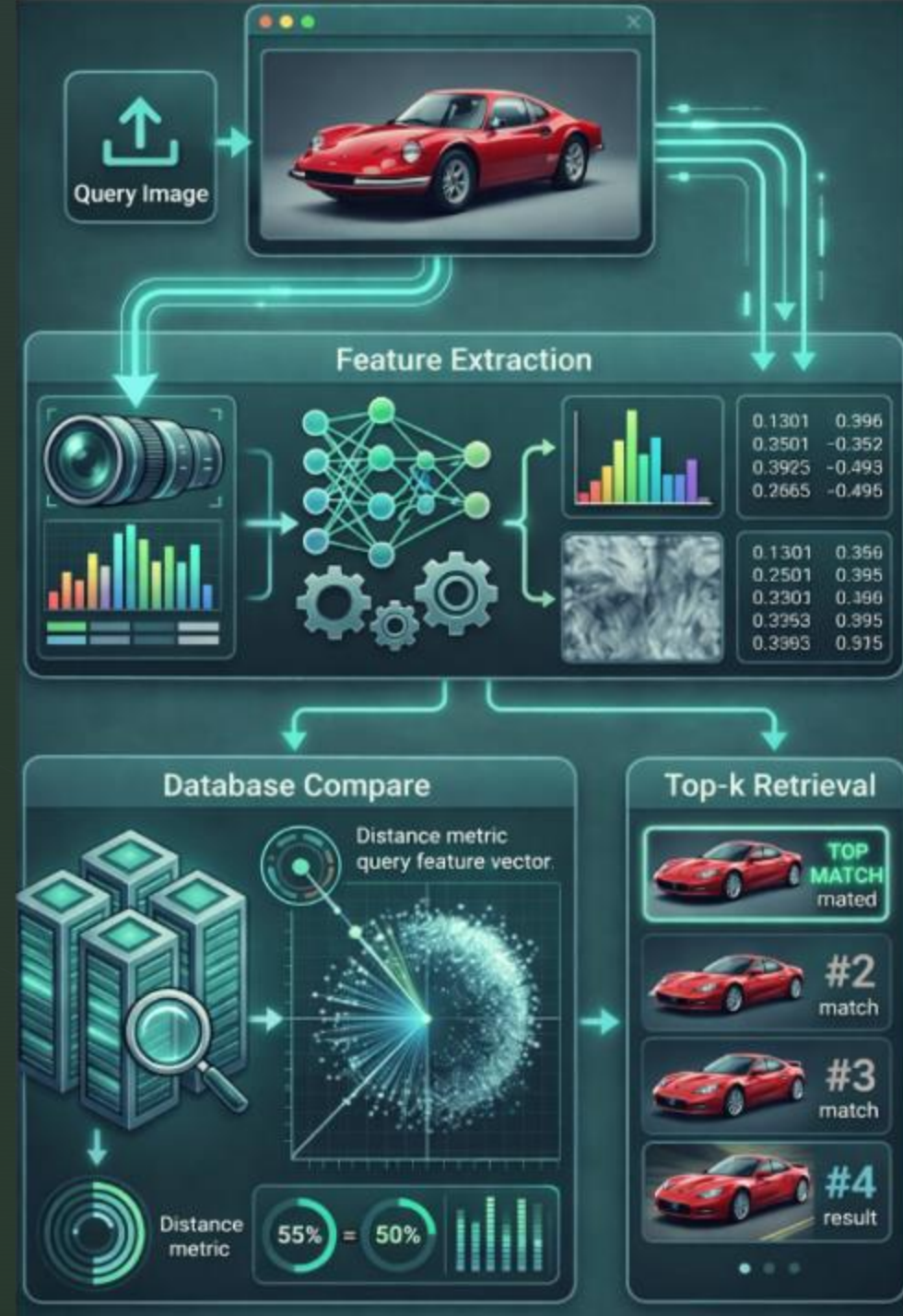
SIMILARITY AND RANKING

- Compute distance between vectors
- Use Euclidean distance
- Rank images by similarity score



QUERY PROCESSING

- Input query image
- Extract features
- Compare with database
- Retrieve top-k results



RESULT VISUALIZATION:

- Display retrieved images
 - Sorted by similarity
- User can explore results



EXTENDED ARCHITECTURE

- Integration with metadata
- Support for textual queries
- Hybrid retrieval (text + image)



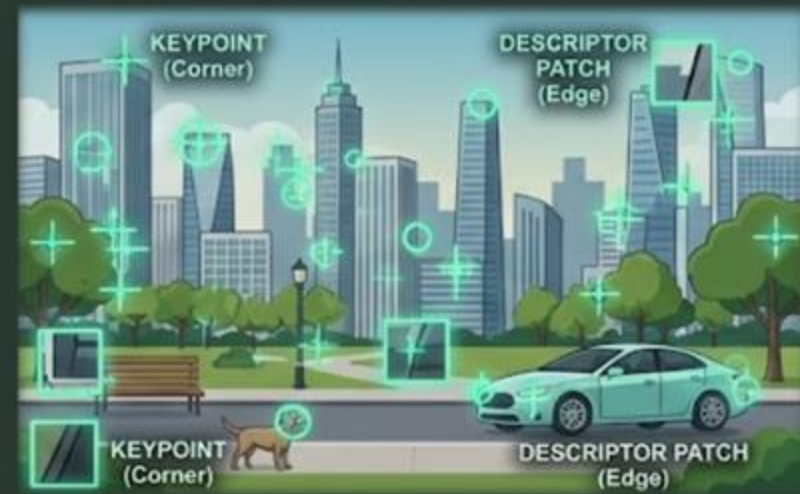
GLOBAL VS LOCAL FEATURES (VISUAL)

GLOBAL



Global Analysis: Holistic Scene Properties
(e.g., Color Histogram)

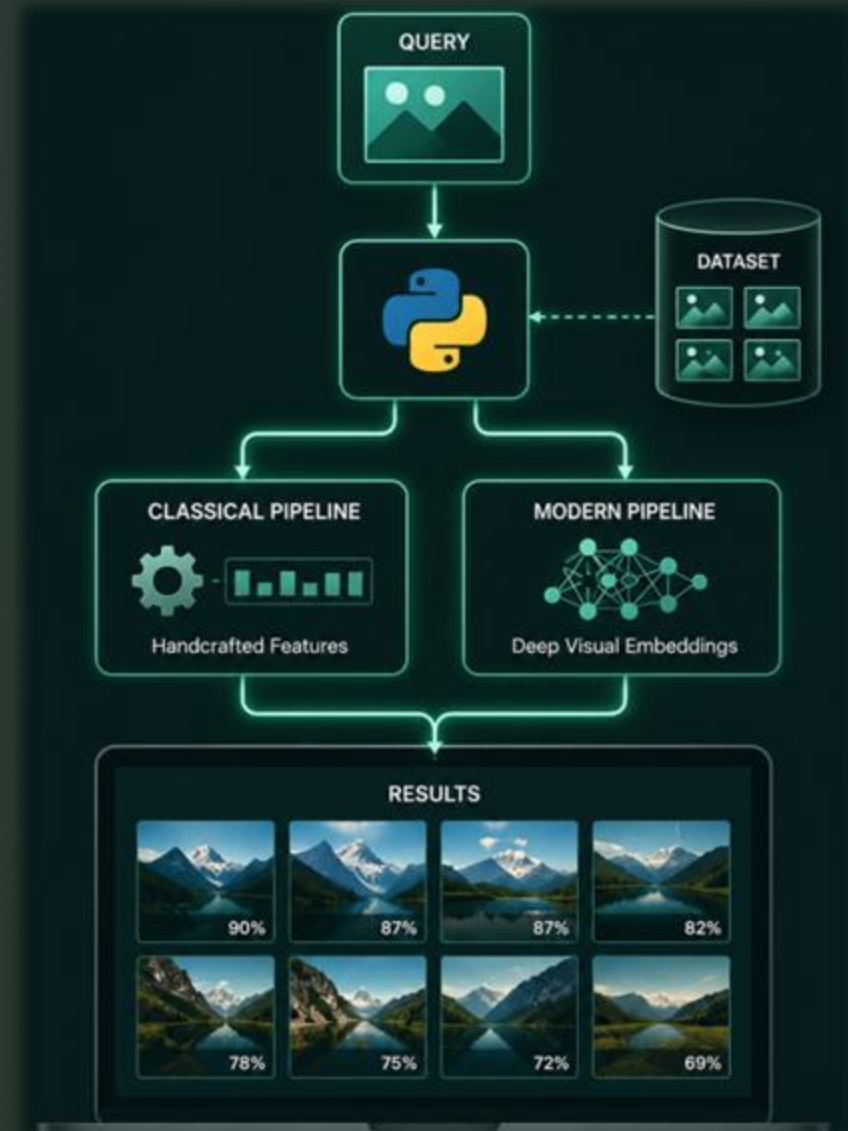
LOCAL



Local Analysis: Distinct Points and Regions
(Descriptors)

IMPLEMENTATION OVERVIEW:

- SmartImageSearch was also implemented as a practical prototype in Python
- The implementation follows two retrieval pipelines on the same dataset
- Classical pipeline based on handcrafted features
- Modern pipeline based on deep visual embeddings



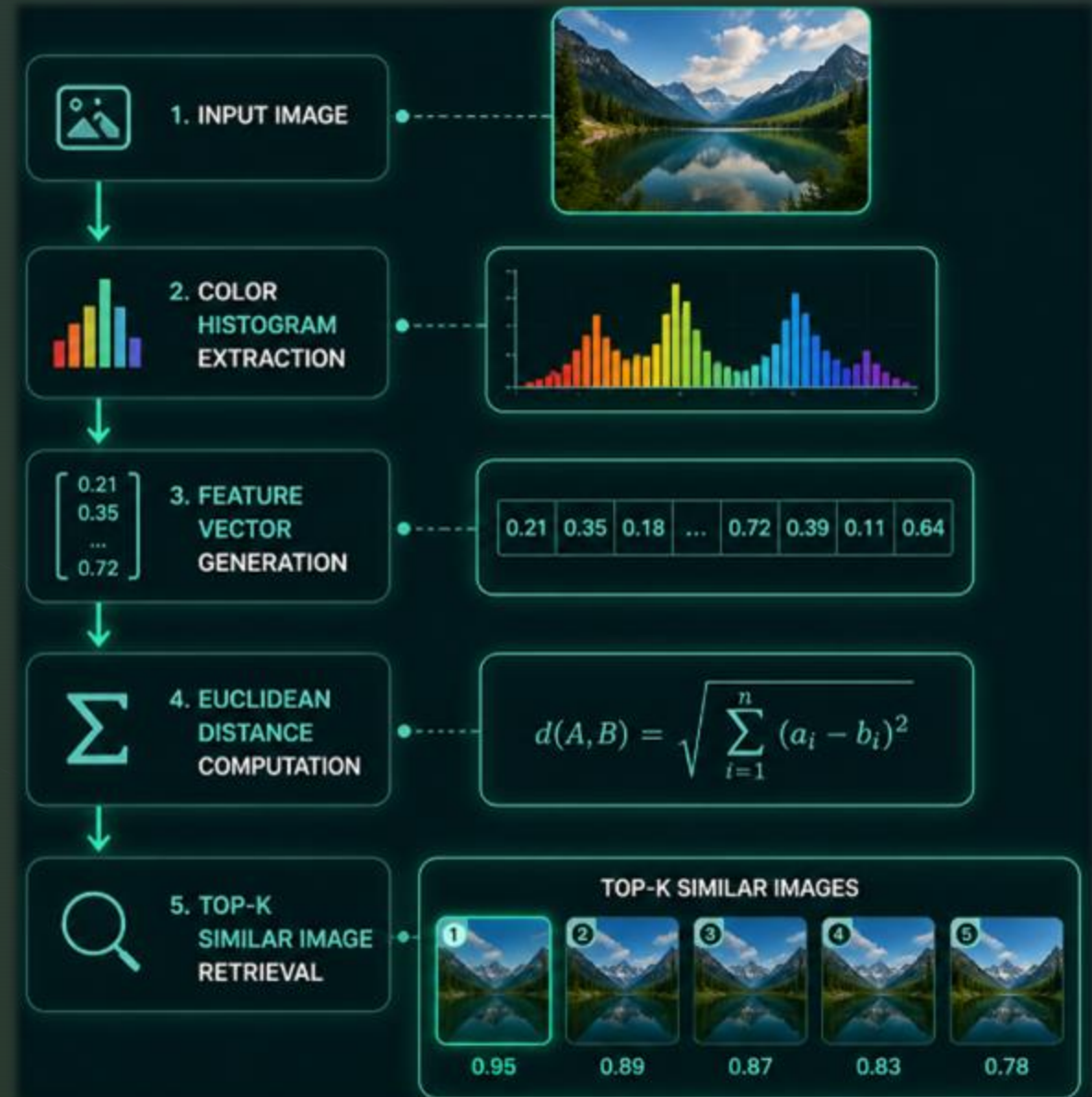
DATASET AND EXPERIMENTAL SETUP

- Dataset: CIFAR-10 subset
- Selected classes: airplane, automobile, cat, dog
- Images used for retrieval experiments
- Same dataset used for both pipelines



CLASSICAL RETRIEVAL PIPELINE

- Input image
- Color histogram extraction
- Feature vector generation
- Euclidean distance computation
- Top-k similar image retrieval

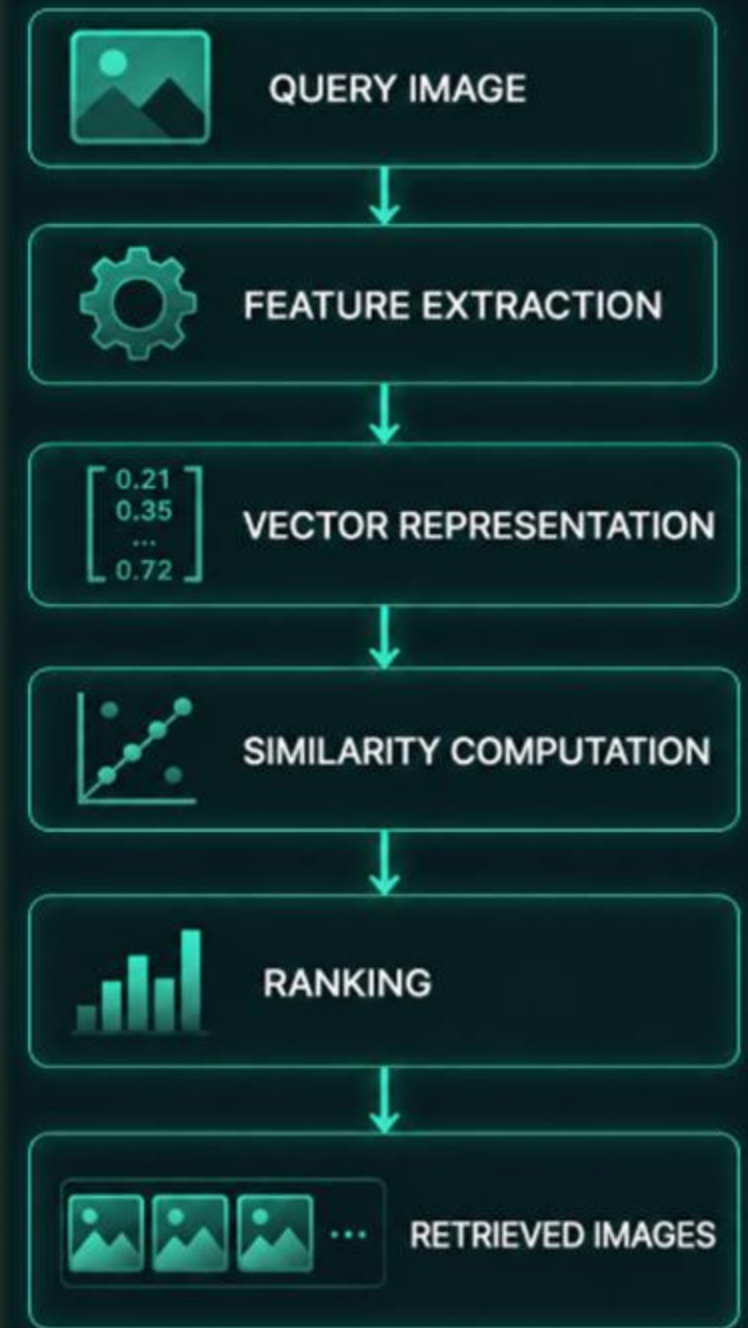


MODERN EMBEDDING-BASED PIPELINE



IMPLEMENTATION WORKFLOW

- Classical pipeline: handcrafted features
- Modern pipeline: learned embeddings



PRACTICAL NOTEBOOK STRUCTURE

- Imports and configuration
- Dataset loading
- Subset selection
- Classical retrieval implementation
- Modern retrieval implementation
- Performance comparison

RETRIEVAL EXAMPLE: CLASSICAL PIPELINE



Query label: automobile



RETRIEVAL EXAMPLE: MODERN PIPELINE

QUERY IMAGE
automobile



Query label: automobile

Modern Embedding-Based Retrieval

automobile
0.8832



automobile
0.8762



automobile
0.8726



automobile
0.8689



automobile
0.8619

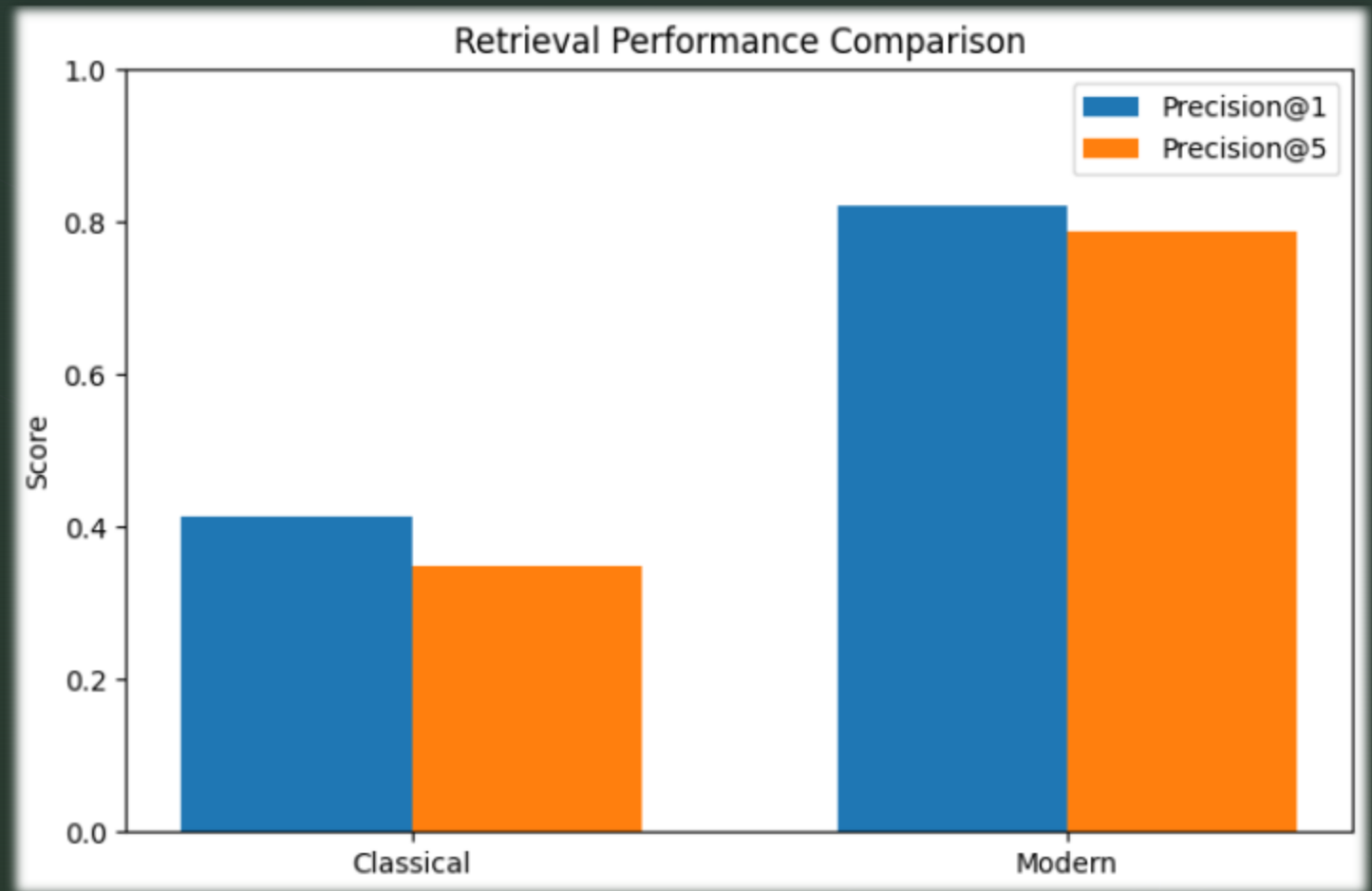


PIPELINE COMPARISON

 CLASSICAL PIPELINE	VS	MODERN PIPELINE 
 Color histogram	↔	 Deep embedding
 Euclidean distance	↔	 Cosine similarity
 Interpretable	↔	 Semantically richer
 Simpler	↔	 More powerful
 Exact similarity on features	↔	 Representation-based similarity

EVALUATION

- Evaluation metric: Precision@5
- Both pipelines tested on the same subset
- Comparison based on retrieval quality



FROM THEORY TO IMPLEMENTATION

- Theoretical feature modeling was translated into executable pipelines
- Classical multimedia retrieval was implemented through handcrafted descriptors
- Modern retrieval was implemented through neural embeddings
- The project shows the evolution from traditional to current multimedia retrieval systems

USE CASES

- Photo organization
- Image search engines
- E-commerce (similar products)
- Visual recommendation systems



■ BENEFITS

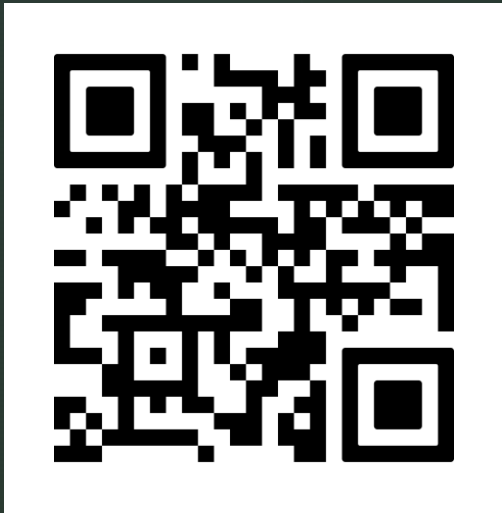
SmartImageSearch

- No need for manual annotation
- Faster image retrieval
- Improved user experience
- Scalable to large datasets

Traditional System

- Requires manual annotation
- Slow and inefficient search
- Depends on textual metadata
- Limited retrieval accuracy
- Not scalable for large datasets

SmartImageSearch GitHub Repository



The complete project is available on [GitHub](#) and includes:

- Project presentation (Slides A—F)
- Jupyter Notebook implementation
- Retrieval examples and visual results
- Project documentation and reproducibility material

CONCLUSION

SmartImageSearch improves multimedia data retrieval by enabling content-based image search using visual features and similarity measures.